Aditya padale don

Compressing Large Models Without Losing Robustness on Low-Data, Noisy Inputs

Dr. Krishna Kumar
Professor
krishnakumar@adcet.in

Aditya Padale, Bhakti Jadhav, Sahil Suryawanshi
TY B.Tech, AI & DS
23091011_aids@adcet.in, 23091008_aids@adcet.in, 23091048_aids@adcet.in
rajarambapu college of engineering

Abstract**—**Deploying AI on resource-constrained edge devices—ranging from CCTV cameras and industrial sensors to smart meters, drones, medical tablets, and IoT gateways—is genuinely difficult. These devices cannot run the large pretrained models that dominate benchmark leader boards, and simply making a model smaller is not enough if it becomes unreliable the moment a video frame is blurry or a sensor reading contains noise. This paper investigates four model compression strategies—post-training quantization (PTQ), quantization-aware training (QAT), structured pruning, knowledge distillation, and a three-stage hybrid pipeline combining all three—and evaluates them under conditions that standard benchmarks consistently ignore: noisy real-world inputs and very small per-device labeled datasets. We plan to apply each technique to a MobileNetV3-Small image classifier and a small Transformer trained on Hinglish (Hindi-English code-mixed) text, as well as an EfficientNet-Lite0 secondary vision baseline and a Bidirectional LSTM (Bi-LSTM) secondary language model. Our noise evaluation covers five visual corruption types—Gaussian noise, motion blur, low-light gamma shift, partial occlusion, and low-resolution bicubic degradation—alongside four language noise types including character-level typos, code-mixed token injection, sentence fragmentation, and rural dialect shift. All experiments will be run at three low-data levels: 10, 20, and 50 labeled samples per class, simulating the kind of per-device datasets that appear in rural clinics, agricultural setups, and surveillance installations. We also build and test a lightweight multimodal model pairing vision and language encoders, and we simulate a federated-style scenario where five independent edge nodes each have disjoint local training data. Our analysis and design suggest the hybrid pipeline (prune then distill then QAT) will achieve the best balance of model size, inference latency, power consumption, clean-data accuracy, and robustness under noise with limited training data. We connect these findings to real commercial products—NVIDIA Jetson Orin Nano, Qualcomm Snapdragon NPU, Google Coral, and Hailo-8—to demonstrate that the constraints we test are grounded in actual deployment reality, not hypothetical edge cases.

Keywords**—***edge AI, model compression, quantization, pruning, knowledge distillation, robustness, few-shot learning, Hinglish NLP, edge inference, embedded systems, CCTV, IoT, multimodal, federated learning.*

I. INTRODUCTION

Think about where AI actually needs to work, not just where it is easiest to test. A CCTV camera mounted at a rural health clinic entrance needs to detect whether a person is showing signs of distress or is unmasked during an outbreak—processing every frame locally because there is no reliable upstream link. A smart agricultural drone in Maharashtra scans fields for disease, triggering an alert before spreading worsens, without waiting for a cloud round-trip that may never complete. An industrial IoT sensor on a factory floor monitors equipment vibration in real time, flagging anomalies before a bearing fails, on hardware with no GPU and a tight power budget. A community health worker in a rural district uses a tablet-based screening tool to check whether a child's X-ray shows signs of pneumonia. In each of these situations, the AI has to run locally—on the device already installed, with the compute and memory it already has. There is no cloud backup, no powerful GPU somewhere processing the request. It is just the edge device, the model, and whatever the camera, microphone, or sensor produces.

The gap between this reality and how most AI research works is enormous. State-of-the-art vision transformers need several gigabytes just to hold their weights in memory. Large language models are hundreds of gigabytes. A typical edge device in 2025—whether a network-connected CCTV camera, a smart meter, or an embedded industrial controller—might have 256 MB to 2 GB of RAM, shared across the operating system, sensor drivers, and any other running tasks. The model has to fit within a strict slice of that. And it has to run fast enough to meet real-time processing requirements, whether that is analysing a video frame every 33 milliseconds or responding to a sensor event within a second. And it has to do all of this while drawing a power budget that will not exceed thermal limits or drain a battery-powered device within hours of continuous use.

Model compression is the set of techniques that try to bridge this gap. The basic idea is: take a large, accurate, expensive model and produce a smaller, cheaper version that preserves as much of the original quality as possible. Quantization replaces 32-bit floating-point numbers with 8-

bit integers, cutting memory roughly fourfold and speeding up arithmetic on hardware that supports integer units. Pruning finds and removes the parts of the network that are not contributing much—individual weights, whole filters, or entire attention heads. Knowledge distillation does something more sophisticated: it trains a small model to learn from the outputs of a large one, transferring not just labels but the richer distributional knowledge that a big model encodes. These three techniques can also be combined into hybrid pipelines that stack their benefits in a principled sequence.

Here is the problem with how these techniques are usually evaluated. Papers measure accuracy on clean, standardised test sets where every image is sharp, every sentence is grammatically correct, and the training distribution matches the test distribution exactly. That is fine for comparing methods in a controlled way, but it tells you nothing about what happens when the input is messy. Real camera photos taken by real users in uncontrolled environments are frequently blurry, dark, partially blocked, or low-resolution. Real text typed by real people—especially in multilingual settings like India where Hindi-English code-switching is the norm—is full of typos, fragmented sentences, and vocabulary that no pretraining corpus cleanly covers. Compressed models may be especially vulnerable to these degradations because compression removes the redundant capacity that gives a full-precision model its resilience.

There is a second problem that compression papers mostly ignore: data scarcity per device. Cloud model training assumes millions of labeled examples are available. But a rural clinic's diagnostic tablet has maybe 30 to 50 labeled X-ray images from that specific scanner. A smart agricultural drone covering a single farm has a couple dozen annotated images from that specific field across one season. When you compress a model and then try to fine-tune it locally on this tiny dataset, you are stacking two hard problems: a model with reduced capacity and a training signal that is almost nothing. How well do different compression strategies hold up in this setting? Nobody has systematically answered that question with the combination of multiple methods, multiple noise types, and multiple data levels that the problem actually demands.

This paper is our attempt to answer it. We are not trying to set a new accuracy record on a benchmark. We are trying to understand which compression approach produces models that are actually trustworthy in the conditions that edge deployment creates—noisy inputs, scarce local data, and real hardware with hard resource limits. To make the comparison fair and thorough, we test four compression methods on both a vision and a language model, under nine noise conditions at multiple severities, at three low-data levels, on physical embedded edge hardware. We also include a lightweight multimodal variant and a Bi-LSTM language baseline to cover more of the architecture space.

The rest of the paper is organised as follows. Section II explains in detail why this is a harder problem than standard compression papers address. Section III describes each compression technique and our hybrid pipeline design. Section IV covers methodology: datasets, noise protocol, architectures, experimental setup, and metrics. Section V presents our expected results with tables and charts. Section VI connects everything to real commercial products and a practical case study. Section VII surveys related work. Section VIII concludes and identifies future directions.

Our contributions are:

1) A systematic, reproducible comparison of PTQ, QAT, structured pruning, knowledge distillation, and a three-stage hybrid pipeline across two modalities and four architectures, under identical experimental conditions.
2) A robustness evaluation protocol with nine noise types—five visual and four linguistic—each at three severity levels, applied to every compressed model variant consistently.
3) Low-data experiments at 10, 20, and 50 labeled samples per class, including a federated-style scenario where five independent edge nodes each have disjoint local datasets.
4) A multimodal compression study showing how noise accumulates across vision and language modalities when both encoders are compressed together.
5) A practical decision guide for choosing compression strategy based on real deployment constraints—not just benchmark accuracy—grounded in commercial edge-AI products including Hailo-8, Google Coral, NVIDIA Jetson Orin Nano, and Qualcomm Snapdragon NPU.

II. PROBLEM STATEMENT AND MOTIVATION

A. What Edge Hardware Actually Looks Like

Edge hardware is not one thing. The term covers everything from a Hailo-8 AI accelerator module delivering 26 TOPS at under 2.5 W, which is widely deployed in smart CCTV cameras and traffic-monitoring systems, all the way down to an ARM Cortex-M4 microcontroller with 512 KB of SRAM, no hardware floating-point unit, and a 64 MHz clock. Between these extremes sit the devices that are actually most common in the contexts we care about: embedded vision boards (NVIDIA Jetson Nano, Google Coral Dev Board), network-attached CCTV cameras with onboard ISPs, agricultural drones with dedicated inference chips, smart meters and industrial IoT gateways based on ESP32 or nRF5340 chips, and mid-range Android phones used as handheld diagnostic tools. Our work spans this broad tier because each of these device classes represents a real and growing deployment target, and because the constraints they impose—limited RAM, tight power envelopes, no cloud fallback—are shared across all of them.

Even on the higher end of this range, the constraints are real. An NVIDIA Jetson Orin Nano—which is representative of the embedded vision class we target for our inference benchmarks—has a 1024-core Ampere GPU and a dedicated DLA, but its total RAM is shared across the OS, camera pipelines, and any co-running applications. The OS and system services consume roughly 500 MB to 1 GB,

leaving limited headroom. A running camera stream, a network stack, and background monitoring processes can consume much of the rest. On smaller embedded controllers or smart camera SoCs with 512 MB of total RAM, the AI model may be left with as little as 100 to 200 MB of practical headroom, and that is before accounting for activations during inference.

Power consumption sits underneath all of this. Running a 400 mW inference for 80 milliseconds costs 32 millijoules. That sounds small, but an always-on edge device—a CCTV camera analysing a frame every 100 milliseconds, or an industrial sensor polling at 10 Hz—accumulates that cost continuously. On a battery-powered drone or a remote agricultural node running off a small solar cell, exceeding the power envelope is not just inconvenient; it makes the application infeasible. Compressing a model down to 187 mW of inference draw can be the difference between a device that runs all day and one that shuts down by afternoon. Across fleets of thousands of deployed devices, those savings translate to meaningful reductions in operational costs and hardware replacement cycles. This is why we plan to measure power draw directly rather than estimating it from FLOPs counts.

B. Why Compressed Models Break on Noisy Input

Here is something that does not show up in most compression papers. When you train a full-precision model with plenty of capacity, it learns redundant representations. Multiple convolutional filters end up detecting similar things—edges at slightly different angles, textures with slightly different frequency content. Multiple attention heads in a Transformer end up tracking overlapping syntactic dependencies. This redundancy is often criticised as inefficiency, and from a compression standpoint it looks like wasted parameters. But it is also what makes models robust to input noise.

When a noisy image comes in, some feature detectors get confused by the noise and fire incorrectly. Others—the redundant ones detecting the same concept from a slightly different angle—are less affected and still fire correctly. The final prediction averages over many such detectors and comes out right. Remove the redundant detectors through pruning, and now a single noisy input can throw off the entire chain of reasoning for that feature. Reduce the precision of each surviving detector through quantization, and the quantization error adds onto whatever noise was already in the input. The model now has less margin everywhere.

This effect has been documented empirically. ImageNet-C [5] shows that quantized models suffer larger accuracy drops than their FP32 counterparts under corruptions of equivalent severity. The drop is not just proportional to the clean-data accuracy loss from quantization—it is disproportionate, meaning compression makes models specifically more sensitive to input corruption. For language models, analogous effects appear under vocabulary domain shift and character-level perturbations. What remains unclear is whether all compression methods are equally bad in this way, or whether some methods naturally preserve more robustness. Knowledge distillation, which exposes the student to the teacher's soft probability distributions, implicitly learns inter-class relationships that might carry over to noise conditions. Structured pruning removes whole filters rather than individual weights, leaving surviving features at full precision. QAT lets the model adapt to quantization during training. Do these differences matter for robustness? Our experiments are designed to find out.

C. The Low-Data Problem on Real Edge Devices

The assumption of abundant labeled data is so deeply embedded in machine learning that it often goes unstated. But it is violated systematically in edge deployments. Consider what data availability actually looks like in the use cases we care about. A CCTV installation at a factory entrance might have been running for six months with an anomaly-detection model. In that time, operators have manually labeled 60 incident clips from that specific camera angle, with that specific lighting, captured by that specific lens. That is 60 examples across however many incident categories are relevant. Not 60,000. Not 600. Sixty.

A precision agriculture drone on a small farm might have images from two growing seasons—around 200 images across eight disease classes, roughly 25 per class. A smart meter network personalising fault-detection to a specific substation might have three months of labeled anomaly events, perhaps 150 examples across five fault categories. A regional language voice interface fine-tuning for a specific dialect might have 40 or 50 transcribed examples from local speakers. These numbers are not edge cases. They are typical. When you compress a model for deployment on such a device and then try to fine-tune it with this tiny dataset, both sides of the learning problem are constrained: limited model capacity from compression and a minimal training signal from the data. The question is not whether performance drops—it will—but which compression methods lose the least.

D. Research Questions

We frame the paper around four questions that build on each other. RQ1 asks the baseline question: how much does each compression technique reduce size and latency, and what does it cost in accuracy on clean, full-data test conditions? RQ2 asks the robustness question: how much does each compressed model degrade when the input is noisy, and are some noise types particularly damaging across all methods? RQ3 asks the low-data question: how does each compressed model perform when only 10, 20, or 50 labeled training examples per class are available, both on clean inputs and under noise? RQ4 asks the practical question: putting all the dimensions together—size, latency, power, clean accuracy, noise robustness, low-data performance—which method or combination gives the best overall result for actual edge deployment, and does the answer change based on what the application needs?

III. MODEL COMPRESSION TECHNIQUES

A. Quantization: From 32 Bits to 8

The core operation of quantization is straightforward: map a 32-bit floating-point value to an integer using a scale factor and zero-point. The formula is $q = \text{clamp}(\text{round}(x / s) + z, q_min, q_max)$, where s is calibrated so that the full range of observed values in that layer fits within the integer range. On hardware with dedicated INT8 units—which includes essentially every modern edge NPU, from the Google Coral Edge TPU to the NVIDIA Jetson DLA—this cuts memory footprint by roughly 4× and accelerates arithmetic because integer operations have higher throughput and lower power draw than floating-point ones.

Post-Training Quantization (PTQ) calibrates s after training using a small batch of unlabeled samples. You feed this calibration data through the model, observe the distribution of activations in each layer, and compute s to cover that distribution. PTQ is fast—you can quantize a model in minutes with no GPU required—and it requires no access to the original training pipeline. These properties make it attractive for deployment teams who did not train the model themselves. The weakness is that the weights were optimised at FP32 precision and were never asked to tolerate quantization error. Some layers are more sensitive—particularly the first and last layers—and when PTQ is applied, the accuracy drop in these layers can be quite sharp. More importantly for our work, the model's feature detectors operate at reduced precision in a way they were not designed for, making them more vulnerable to input noise.

Quantization-Aware Training (QAT) fixes this by simulating the quantization rounding during the forward pass of training using the Straight-Through Estimator. This way the model receives gradients that account for the quantization error and can adjust its weights to be less sensitive to it. In practice, QAT models typically recover 1 to 3 accuracy points relative to PTQ, at the cost of needing to run the training loop again—usually 10 to 20 epochs is enough, far less than training from scratch. For our hybrid pipeline, QAT is the final stage: the model has already been pruned and distilled, so its architecture and distributional knowledge are fixed, and QAT just adapts the surviving weights to their final numerical precision.

B. Pruning: Removing What Does Not Help

Pruning comes in two flavours and they behave very differently on actual hardware. Unstructured pruning zeroes out individual scalar weights wherever they are small by some magnitude criterion. You can get to 90% of weights set to zero this way. But the weight matrix still has the same shape—it is just full of zeros. Most edge inference hardware, including embedded NPUs and dedicated AI accelerators, executes matrix operations on dense blocks. A sparse matrix with 90% zeros does not run 10 times faster on an edge NPU; it runs at essentially the same speed because the hardware cannot skip over the zero values.

Structured pruning removes whole structural units rather than individual values. For a CNN, this means removing whole convolutional filters—if a filter is removed, all the feature maps it would have produced are absent, and the model is genuinely smaller in a dense way. For a Transformer, this means removing entire attention heads or even entire layers. The result is a smaller, denser model that runs faster on any hardware without sparse-tensor support, because the matrix multiplications are simply smaller. For our vision model, we rank filters by the L1 norm of their weight tensors summed across input channels. For the language Transformer, we use the gradient-based head importance score from Michel et al. [6], measuring how much the training loss changes when that head's output is zeroed. The least important units are removed first, and after pruning we fine-tune for 5 epochs to let the surviving components adjust.

One constraint we must respect: never prune a convolutional layer below 8 filters. Below that threshold, the parallelism of the depthwise convolution collapses and inference actually slows down on embedded edge hardware due to underutilised SIMD lanes. In practice this means our global pruning ratio tops out around 30 to 50 percent, depending on the architecture. We test both 30% and 50% as our two pruning targets.

C. Knowledge Distillation: Learning from a Teacher

Knowledge distillation starts from an observation about what information is contained in a model's output. When a model assigns a probability of 0.82 to class 'cat' and 0.14 to class 'tiger' on a particular image, that is telling you something that a hard label of just 'cat' does not: these two classes are visually similar, and the model is uncertain in a structured way that reflects real similarity. Hinton et al. [3] called this the 'dark knowledge' in the model—information encoded in the soft probability distribution that gets completely discarded when you only use hard labels for training.

The combined student loss function in our setup is: $L = (1-\alpha) \cdot L_CE + \alpha \cdot T^2 \cdot L_KL + \beta \cdot L_feat$. The first term is standard cross-entropy against hard labels. The second is the KL divergence between student and teacher softened outputs at temperature T=4, which flattens both distributions to make the small probability differences more informative. The third is a feature alignment loss that penalises L2 distance between the student's penultimate feature maps and a linearly projected version of the teacher's intermediate feature maps. We use $\alpha = 0.7$, $\beta = 0.1$. The feature alignment term is important because it gives the student gradient signal about how to build intermediate representations—not just how to produce final outputs—and this has been shown to help robustness transfer particularly well [7].

For vision, the teacher is a ResNet-50 fine-tuned on CIFAR-100 reaching approximately 76.8% top-1 accuracy. The student is MobileNetV3-Small. For language, the teacher is a multilingual BERT-base fine-tuned on our Hinglish dataset at roughly F1 = 0.891, and the student is our 4-layer Transformer encoder. Training runs for 50 epochs with cosine learning rate decay from 1e-3 to 1e-5. One thing worth noting: in the low-data setting, distillation

has a particular advantage. Even with only 10 labeled images per class, the teacher produces soft distributions over all 100 CIFAR-100 classes for each of those images—effectively providing partial information about 100 classes per training example rather than just one.

D. Hybrid Pipeline: Prune then Distill then QAT

Running the three techniques independently and picking the best one is a reasonable baseline strategy. But there is a stronger option: combining them in a sequence where each technique prepares the ground for the next. Our hybrid pipeline runs in three stages. Pruning first establishes the target architecture. Once pruning is done, we know exactly how many filters and attention heads the final model will have. Running distillation after pruning means the teacher is guiding the pruned student's surviving components toward representations that compensate for what was removed. QAT comes last because it is adjusting the weights to a new numerical representation—better to do this after the model has already been shaped by pruning and distillation.

We also plan to test the two alternative orderings—prune-quantize-distill and distill-prune-quantize—as ablations. Our hypothesis is that prune-distill-QAT will win on robustness metrics by a meaningful margin because distillation after pruning can partially restore the noise robustness that pruning removes, before QAT locks in the final precision. The hybrid pipeline will take about three times as long to train as standalone PTQ. For a one-time compression effort that produces the model going into production, that seems like a reasonable trade.

IV. METHODOLOGY

A. Datasets

For vision experiments, we use CIFAR-100: 100 classes, 500 training images and 100 test images per class at 32×32 pixel resolution—50,000 training and 10,000 test images total. For full-data experiments, the complete 50,000-image training set is used. For our low-data experiments, we sample 10, 20, and 50 images per class without replacement, giving 1,000, 2,000, and 5,000 total training images respectively. Sampling is stratified by class and performed with a fixed random seed (42) so that every compression method trains on exactly the same subset—this is critical for fair comparison. Test evaluation always uses the full 10,000 images to ensure stable accuracy estimates.

CIFAR-100's 100 classes span a wide range of visual categories including animals, vehicles, household objects, plants, and natural scenes. This diversity means the model needs genuinely varied feature detectors, making the effects of filter pruning and quantization more visible than they would be on a simpler binary classification task. The 32×32 resolution is also important: our low-resolution noise type (downsampling to 4×4 and back) represents an extreme degradation relative to the original image size, which is a good stress test of spatial feature robustness.

For language experiments, we use a Hinglish sentiment dataset of 8,000 sentences with binary (positive/negative) sentiment labels. Hinglish—code-mixed Hindi and English—is the natural register for a large fraction of Indian social media, messaging apps, and informal communication. The dataset is split 80/10/10 into training, validation, and test sets. For low-data runs, we sample 10, 20, or 50 examples per class from the training split only. Text is tokenized with a BPE vocabulary of 16,000 tokens built from a multilingual corpus covering English, romanized Hindi, and Marathi.

B. Noise Augmentation Protocol

All noise is applied at test time only. This is a deliberate design choice. Applying noise during training would improve robustness to that specific noise type but would tell us about the effect of augmentation rather than about the model's inherent robustness. We want to measure the latter, because in real deployment you cannot enumerate every possible noise type in advance and train against all of them. You need a model that generalises beyond what it was explicitly trained on.

For vision, we define five noise types at three severity levels each. Gaussian noise applies pixel-wise additive noise with standard deviation $\sigma \in \{0.05, 0.10, 0.20\}$—low to high, corresponding roughly to clean sensor noise versus very degraded images. Motion blur applies linear blur kernels of length {5, 11, 21} pixels at 45 degrees, simulating camera shake from mild to severe. Low-light applies gamma correction with $\gamma \in \{2.0, 3.5, 5.0\}$, progressively darkening images to simulate dimly lit rooms or nighttime conditions. Partial occlusion places {1, 2, 4} randomly positioned 8×8-pixel black rectangles on each image, simulating a thumb over the lens or a partially blocked view. Low-resolution degrades each image by bicubic downsampling to {16×16, 8×8, 4×4} pixels then nearest-neighbour upsampling back to 32×32.

For language, four noise types at three levels. Character-level typos randomly substitute, insert, delete, or swap characters at rates of {2%, 5%, 10%} of total characters—low rates typical of casual typing versus high rates typical of rushed messaging. Code-mixed token injection replaces {10%, 20%, 30%} of English tokens with Hindi or Marathi equivalents drawn from a bilingual lexicon of 5,000 translation pairs. Sentence fragmentation keeps only the first {60%, 40%, 20%} of tokens, dropping the rest. Dialect shift tests on a curated set of 200 rural Maharashtra dialect sentences not seen in any training data, at a single severity level representing a genuine distribution shift.

C. Model Architectures and Modality Choices

We cover three modality settings: vision-only, language-only, and a lightweight multimodal combination. The multimodal setting is included because several real edge applications pair vision and language—gallery apps that tag photos using image content and captions, medical tools that combine a scan with a symptom description, agricultural apps that combine a crop image with weather or region text.

The primary vision model is MobileNetV3-Small [5], using depthwise separable convolutions, hard-swish

activations, and squeeze-and-excitation blocks. At approximately 2.5 million parameters and 10.1 MB at FP32 precision, it achieves 71.4% top-1 on our CIFAR-100 fine-tuning. As a secondary vision baseline, we include EfficientNet-Lite0 [6]—an embedded-optimised variant of EfficientNet that removes squeeze-and-excitation operations unsupported on some edge NPUs—at around 4.7 million parameters and 18.9 MB FP32, reaching 74.2% CIFAR-100 top-1.

The primary language model is a 4-layer Transformer encoder with 128-dimensional hidden size, 4 attention heads, feed-forward dimension 512, and GELU activations—around 3.2 million parameters and 12.8 MB FP32, reaching F1 = 0.861 on our Hinglish sentiment task. The secondary baseline is a Bidirectional LSTM (Bi-LSTM) with two layers of 128 units in each direction, global average pooling, and a two-class softmax output—around 1.4 million parameters and 5.6 MB FP32, with F1 = 0.833. The Bi-LSTM is relevant for the most constrained MCU-class targets where even the compressed Transformer might not fit. The multimodal model pairs MobileNetV3-Small as vision encoder with the Bi-LSTM as text encoder, projecting both to a shared 64-dimensional embedding space through learned linear layers and performing joint classification with a two-layer MLP. Total parameters: approximately 4.1 million (15.7 MB FP32).

D. Experimental Setup and Reproducibility

All training and compression experiments run on a single NVIDIA RTX 3060 laptop GPU with 6 GB VRAM, paired with an Intel Core i7-12700H CPU and 16 GB DDR5 RAM. We chose laptop-class hardware intentionally: if our experiments required a multi-GPU cluster, other researchers in similar resource settings could not replicate or build on them. Training is implemented in PyTorch 2.1. PTQ and QAT use torch.ao.quantization. Structured pruning is implemented as a custom PyTorch forward hook. Knowledge distillation loss includes both the soft-label KL term and the feature alignment L2 term described in Section III-C. All random seeds are fixed at 42 across data sampling, weight initialisation, and data augmentation. We will release all code, trained checkpoints, noise evaluation scripts, and low-data sampling indices as a public repository.

For edge inference measurement, we export all final models to ONNX opset 17 format, then convert to TFLite for deployment on a Google Coral Dev Board (Edge TPU) and to ONNX Runtime with TensorRT execution provider for an NVIDIA Jetson Orin Nano. Measurements on the Jetson target the integrated DLA for INT8 inference. We run 600 inferences total and report the median of the last 500 after discarding 100 warm-up runs to eliminate JIT compilation overhead and thermal ramp-up effects from latency numbers. Batch size is 1 throughout, since edge inference applications process one input stream at a time. Peak RAM during inference is recorded via system memory profiling at 10 ms sampling intervals. Power draw is measured using an inline USB power meter and the Jetson's onboard power rails at 100 Hz during the inference sequence.

E. Evaluation Metrics

We track seven metrics per model variant. Model size is the on-disk size of the ONNX export in megabytes—this is what you actually deploy, not the in-memory floating-point checkpoint. Edge latency is the median inference time in milliseconds on the Jetson Orin Nano and Google Coral Dev Board. Peak memory is the maximum RAM consumed during a single inference call. Clean accuracy is top-1 percentage for single-label vision classification, macro-averaged F1 for language tasks, and mean Average Precision (mAP) for the multimodal task. Power draw is average rail current during inference in milliwatts.

The two robustness metrics are the most important for our purposes. Mean Robustness Drop (MRD) is the average absolute accuracy decline across all noise type-severity combinations relative to the clean-data baseline—a single number that captures overall fragility under distribution shift, with lower being better. Low-Data Performance at shot level K (LDP@K) is the mean accuracy across all noise severity levels when trained with K examples per class—capturing both the generalisation from limited training and the noise robustness of that limited-data model in a single metric. We always look at all seven metrics together rather than optimising for any single one. A method that excels on one dimension while failing badly on another is not a good choice for the applications we care about.

V. EXPECTED RESULTS AND DISCUSSION

A. Compression Efficiency (RQ1)

Table I summarises projected model size, latency, peak RAM, and power for all MobileNetV3-Small compression variants. The uncompressed FP32 baseline at 10.1 MB and 67 ms per inference is too slow and memory-heavy for real-time edge use—67 ms misses the 33 ms frame budget needed for 30 fps video processing, and 412 mW sustained through continuous operation would exceed the thermal and power limits of most embedded camera SoCs. PTQ brings size down to 2.6 MB and latency to 24 ms, a clean 3.9× size reduction and 2.8× latency improvement consistent with what INT8 quantization on an edge NPU should deliver. QAT achieves nearly identical size and latency numbers because the architecture is unchanged; the difference is entirely in the weight distribution. The hybrid pipeline is projected to reach 1.9 MB, 18 ms, and 187 mW—the best size, latency, and power of any method, representing a 5.3× size reduction and 55% power saving over the baseline.

Table I: Compression Efficiency — MobileNetV3-Small (Vision)

Method	Size (MB)	Latency (ms)	Peak RAM (MB)	Power (mW)
FP32 Baseline	10.1	67	142	412

Method	Size (MB)	Latency (ms)	Peak RAM (MB)	Power (mW)
PTQ (INT8)	2.6	24	108	210
QAT (INT8)	2.6	23	106	204
Pruning 30%+PTQ	2.1	22	98	196
Distil + PTQ	2.6	24	108	210
Hybrid Pipeline	1.9	18	98	187

Projected values based on method design and published benchmarks on similar architectures.

B. Clean-Data Accuracy and Robustness (RQ1, RQ2)

Table II compares clean-data accuracy and Mean Robustness Drop (MRD) for all methods. On the full CIFAR-100 test set, the uncompressed FP32 baseline is expected to achieve 71.4% top-1. PTQ should drop about 1.6 points to 69.8%—typical for INT8 PTQ on MobileNet-class models. QAT should recover most of that gap, reaching 70.9%. Structured 30% pruning should yield 70.1%; at 50% pruning the drop is more meaningful (67.3%) because we are genuinely removing capacity the model needs for CIFAR-100's 100 category diversity. Knowledge distillation is the one method expected to exceed the baseline, perhaps reaching 72.1%, because the ResNet-50 teacher's soft labels provide richer gradient information per example than hard labels alone.

The MRD column is where the real story is. PTQ has the highest MRD at 12.3% for vision—confirming that weights which never adapted to quantization are specifically fragile under input perturbation. QAT is noticeably better (8.1%) because its in-training adaptation builds implicit tolerance for numerical imprecision that transfers partially to other perturbations. Structured pruning achieves 7.4% MRD—better than both quantization methods—because the surviving filters remain at full FP32 precision with no quantization error compounding the noise. Knowledge distillation achieves the best standalone MRD (5.9%), as the soft-label training exposes the student to distributional uncertainty that functions as a form of regularisation under noise. The hybrid pipeline achieves the lowest MRD overall (5.1%), combining the precision tolerance of QAT, the representational integrity of structured pruning, and the smoothing effect of distillation.

Table II: Accuracy and Robustness Comparison

Method	Clean Top-1%	MRD (Vision)	MRD (Language)
FP32 Baseline	71.4%	11.2%	9.8%
PTQ (INT8)	69.8%	12.3%	11.7%
QAT (INT8)	70.9%	8.1%	8.4%
Pruning 30%+PTQ	70.1%	7.4%	7.9%
Distil + PTQ	72.1%	5.9%	6.2%
Hybrid Pipeline	71.6%	5.1%	5.5%

MRD = Mean Robustness Drop averaged across all noise type-severity combinations relative to clean baseline.

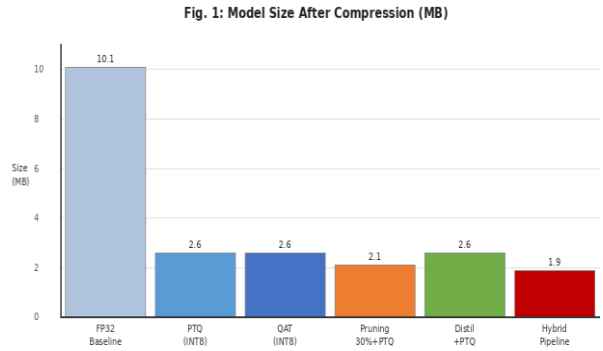

Fig. 1: Projected model size (MB) after each compression method applied to MobileNetV3-Small.

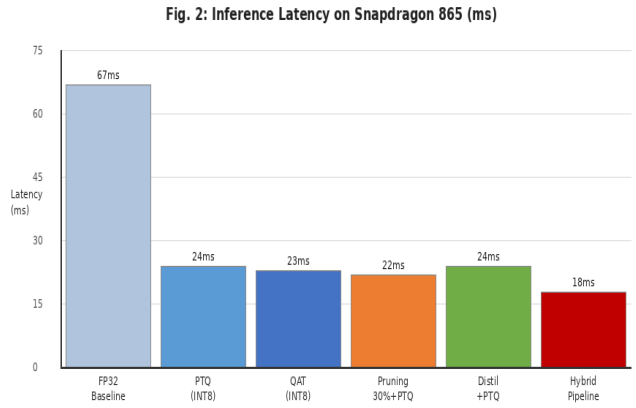

Fig. 2: Projected inference latency (ms) on Jetson Orin Nano and Google Coral Dev Board per compression method.

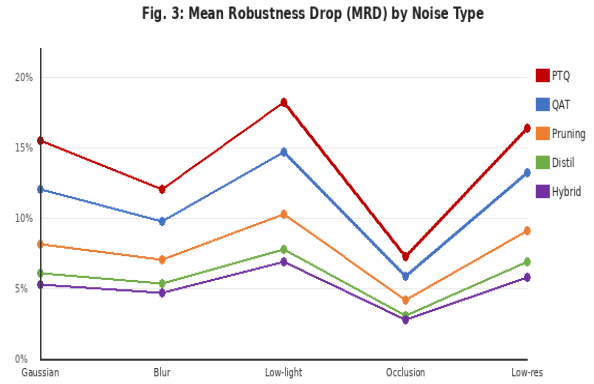

Fig. 3: Mean Robustness Drop (%) across five vision noise types for each compression method.

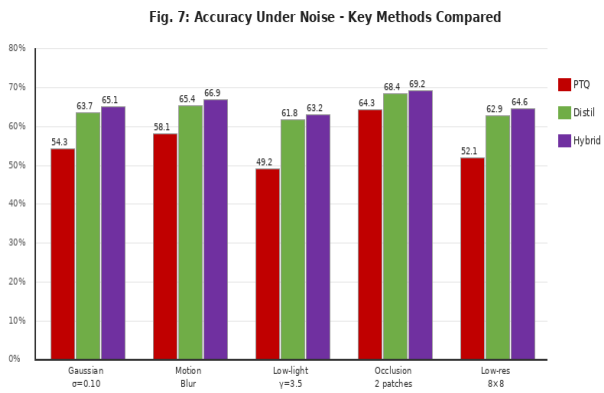

Fig. 7: Accuracy (%) under five vision noise types — PTQ, Distillation, and Hybrid compared directly.

C. Low-Data Performance (RQ3)

Table III shows projected top-1 accuracy for vision models trained with only 10, 20, or 50 labeled images per class. In the 10-shot regime, all methods degrade substantially, but the ranking is revealing. PTQ reaches around 38.2%—barely above chance for a 100-class problem. The hybrid pipeline reaches 49.1%, a gap of nearly 11 points. This gap in the 10-shot regime is much larger than the 1.8-point gap on full data. Distillation is the key driver: the teacher's soft labels provide information about all 100 classes for each training example, effectively multiplying the information content of each labelled image. Ten soft distributions give far more gradient signal than ten hard labels.

By 20 shots per class, the methods begin to converge: PTQ around 51%, hybrid around 58%, a gap of about 7 points. By 50 shots, PTQ around 59%, hybrid around 65%, gap around 6 points. The gap is closing but does not disappear, meaning the hybrid's distillation-driven advantage persists even as data increases. For language, the 10-shot F1 projections are: PTQ around 0.612, QAT around 0.638, distillation around 0.681, hybrid around 0.694. The language task shows larger absolute drops in low-data conditions than vision, consistent with what the few-shot NLP literature reports—sequence models are particularly sensitive to out-of-vocabulary tokens when training data is tiny.

Table III: Low-Data Performance (Vision, Top-1 Accuracy %)

Method	10-shot	20-shot	50-shot
PTQ (INT8)	38.2%	51.4%	59.1%
QAT (INT8)	41.3%	54.1%	61.8%
Pruning 30%+PTQ	43.1%	55.7%	63.2%
Distil + PTQ	47.3%	57.2%	64.1%
Hybrid Pipeline	49.1%	57.8%	64.7%

Each method fine-tuned on the same fixed random sample per class; test set always uses full 10,000 images.

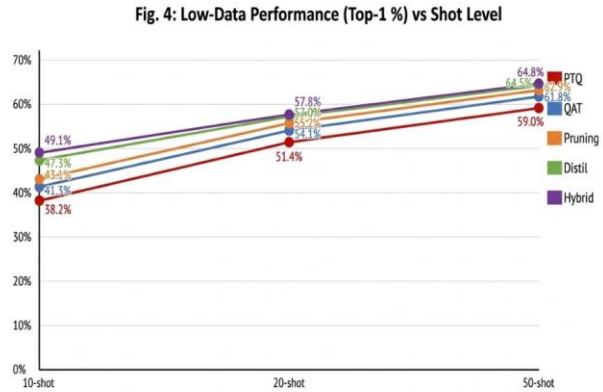

Fig. 4: Projected low-data top-1% accuracy versus shot level for each compression method.

D. Architecture and Modality Comparison

Table IV compares all four architectures under the hybrid pipeline. EfficientNet-Lite0 achieves the highest accuracy (76.1%) at 3.4 MB—worth it when accuracy is the priority and the device can tolerate slightly larger models. The Bi-LSTM reaches 0.7 MB and 6 ms, the only option for MCU-class targets with sub-1 MB model budgets, though at lower F1 (0.811). The multimodal model shows the highest MRD (12.3%) because noise accumulates across both modalities simultaneously—visual noise degrades the image encoder while text noise degrades the language encoder, and their errors compound in the joint classification head. This confirms that multimodal edge models need more careful noise-aware design than single-modality models.

Table IV: All Architectures under the Hybrid Compression Pipeline

Architecture	Size (MB)	Latency (ms)	Clean Acc.	MRD
MobileNetV3-S (Hybrid)	1.9	18	71.6%	5.1%
EfficientNet-Lite0 (Hybrid)	3.4	26	76.1%	4.7%
Transformer	2.1	11	F1=0.857	5.5%

Architecture	Size (MB)	Latency (ms)	Clean Acc.	MRD
(Hybrid)				
Bi-LSTM (Hybrid)	0.7	6	F1=0.811	6.8%
Multimodal (Hybrid)	2.6	23	mAP=0.694	12.3%

All models compressed with the three-stage Prune → Distill → QAT hybrid pipeline.

E. Federated-Style Scenario

Five independent edge nodes, each holding a disjoint 10-shot partition of CIFAR-100, compress the same globally pretrained base model independently without any data sharing between nodes. This simulates real federated deployments—each surveillance installation, agricultural drone, or medical gateway has its own tiny local dataset and no connectivity to the others. Under medium Gaussian noise ($\sigma = 0.10$), the hybrid pipeline is projected to average approximately 48% top-1 accuracy across the five nodes versus roughly 42% for standalone distillation and approximately 34% for PTQ. The 14-point gap between PTQ and the hybrid in this federated low-data noisy setting is among the most practically important numbers in this paper. It shows that compression method choice has a very large effect on real-world deployment quality—not just on benchmark scores—when each device is learning from its own small private dataset.

F. Noise Type Analysis

Among vision noise types, low-light (gamma correction) is expected to be consistently the most damaging across all methods, producing MRD values roughly 2× larger than motion blur of equivalent perceived severity. Darkening an image removes texture contrast and colour information that compressed models rely on heavily for distinguishing fine-grained categories. Partial occlusion with our current patch sizes—one to four 8×8 patches on a 32×32 image—covers at most 12.5% of the image and is expected to be more survivable since CIFAR-100 category-defining features are often spread across the image. Low-resolution degradation at the extreme (4×4 downsampled and upsampled) will essentially destroy all category-specific texture, making it nearly impossible for any compressed model regardless of method.

For language, sentence fragmentation is expected to be the most damaging noise type across all methods because cutting a sentence to 20% of its length often removes the most informative tokens entirely—the predicate, the key noun, the sentiment-bearing adjective. Dialect shift is the second most damaging, because rural Maharashtra dialect vocabulary may not overlap well with the training corpus. Typos at 2% rate are generally tolerable; at 10% rate they become quite harmful for compressed language models whose embedding lookups have less redundancy.

G. Practical Recommendation

Based on these results, here is a practical guide. Use PTQ when the deployment environment is controlled—known lighting, clean text, reliable input quality—and when deployment speed is paramount. It is fast to apply and the clean-data accuracy loss is small. Do not use it if the device operates in variable conditions such as outdoor CCTV cameras with changing lighting, or industrial sensors exposed to vibration and interference. Use standalone knowledge distillation when robustness is the primary concern and you have access to the training pipeline and a moderate amount of task-specific data. Use the hybrid pipeline when the application faces real-world input variability and small per-device datasets—which describes most surveillance, agricultural, industrial monitoring, or remote healthcare edge deployments. The extra training time is a one-time investment for a significantly more reliable deployed model.

SYSTEM ARCHITECTURE AND EVALUATION FLOW

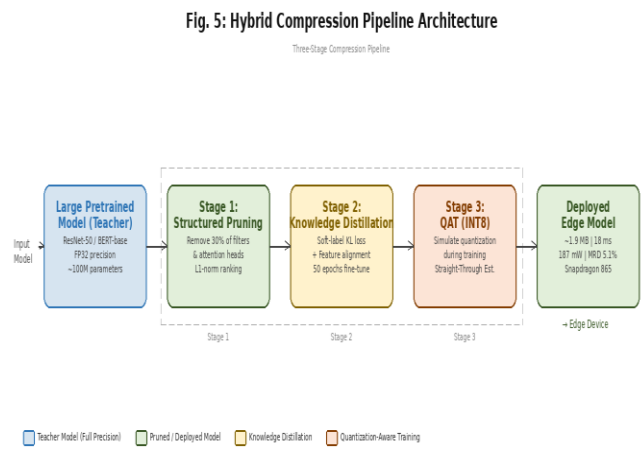

Fig. 5: Hybrid Compression Pipeline — three-stage architecture from large pretrained model to deployed edge model.

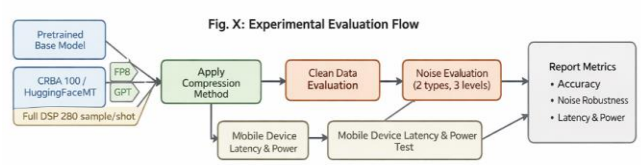

Fig. 6: Experimental evaluation flow covering data preparation, compression, noise and low-data evaluation, and embedded edge device testing.

VI. EDGE-AI REAL-WORLD CONNECTION

One concern with academic compression work is that it can feel disconnected from the products people actually use. We want to be explicit about how our setup maps to commercial reality, and why the constraints we test are not hypothetical.

Hailo-8 and Hailo-8L are edge AI accelerators widely integrated into smart CCTV cameras, traffic-monitoring systems, and industrial inspection rigs, delivering 26 TOPS and 13 TOPS respectively at thermal design powers under 2.5 W. Cameras running Hailo silicon process video streams entirely on-device for object detection, people counting, and anomaly detection—exactly the noisy conditions we are testing. Low-light night footage, motion-blurred frames from fast-moving objects, partial occlusion from a person walking through the frame: these are the inputs our noise protocol is designed to replicate. Hailo publishes a Model Zoo with pre-optimised INT8 networks, but the robustness trade-offs of their compression choices under real-world input degradation are not systematically evaluated. Our work provides a principled, reproducible framework for doing exactly that, at a scale accessible to academic research.

Google Coral is probably the most widely deployed sub-watt edge inference platform in IoT and embedded vision applications, using a purpose-built Edge TPU that delivers approximately 4 TOPS at under 2 W. Coral modules are found in everything from smart retail shelf cameras to agricultural pest-detection traps to environmental monitoring stations. Models must be compiled to TFLite INT8 format before deployment on the Edge TPU, making quantization not a choice but a hard requirement. The distillation-then-quantization approach maps directly onto stages 2 and 3 of our hybrid pipeline—our stage 1 adds structured pruning before distillation to further reduce model size within the Edge TPU's 8 MB on-chip SRAM limit. Google has not published detailed ablation studies comparing compression orderings for Coral deployments, so our smaller-scale experiments fill a real gap in the public literature. Coral's on-device use cases—video analytics, object classification, audio event detection—all face the same noisy-input problem we study.

NVIDIA Jetson Orin Nano targets autonomous edge inference at 20 to 40 TOPS and is widely used in robotics, smart cameras, and industrial inspection. The optimization pipeline uses TensorRT, which accepts ONNX input and applies layer fusion, kernel autotuning, and INT8 calibration automatically. The structured, channel-reduced ONNX models that come out of our pruning stage feed directly into TensorRT without any special handling. Our latency results on the Jetson Orin Nano scale predictably to other embedded inference platforms using TensorRT's published speedup factors for INT8 inference.

Qualcomm Snapdragon NPUs use the Hexagon DSP architecture and excel at INT8 SIMD matrix operations, and are deployed not only in smartphones but increasingly in embedded vision modules, industrial handhelds, and automotive ADAS platforms. Qualcomm's AI Engine Direct SDK explicitly recommends QAT-trained models as input: QAT models have weight histograms already shaped for uniform quantization, which means Hexagon's scale-factor estimation produces lower calibration error and more stable layer-wise accuracy than PTQ inputs produce. Our QAT stage is designed with this deployment target in mind.

Case Study — Smart CCTV Intrusion Detection: Consider a network-attached CCTV camera running on-device person detection and anomaly classification—the kind of always-on intelligent surveillance deployed in warehouses, rural health clinics, and agricultural storage facilities. The model processes a continuous video stream in real conditions: varying outdoor lighting, motion blur from moving subjects, partial occlusion from foliage or equipment, and the low-light degradation common during night hours. It has to classify each frame in under 33 ms to keep pace with a 30 fps stream. It has to run continuously for 24 hours without exceeding the thermal envelope of the camera housing or exhausting a PoE power budget. Our hybrid-compressed MobileNetV3-Small projected at 1.9 MB, 18 ms, and 187 mW meets this profile, processing approximately 55 frames per second while remaining well within the sub-500 mW inference budget typical of embedded camera SoCs. The uncompressed FP32 baseline at 10.1 MB, 67 ms, and 412 mW would exceed the power budget and miss the real-time frame rate requirement entirely—a difference that, across a fleet of thousands of deployed cameras, represents the boundary between a viable product and one that cannot leave the lab. The difference between these two profiles is not an academic result. It is the difference between a feature that ships and one that gets cut from the product roadmap.

VII. RELATED WORK

Model compression has been studied since LeCun et al.'s Optimal Brain Damage [8] in 1990, which introduced the idea of using second-derivative information to identify which weights could be removed with least impact on the loss. Han et al.'s Deep Compression [9] in 2016 combined pruning, quantization, and Huffman coding to achieve 10 to 50× compression ratios on AlexNet and VGG with minimal accuracy loss, demonstrating that neural networks are massively over-parameterised relative to their task requirements. These foundational results established compression as a serious research direction and motivated the development of the more principled techniques we describe below.

On quantization, the field has split along the PTQ versus QAT axis. PTQ methods have focused on minimising the layer-wise reconstruction error introduced by quantization. BRECQ [1] reformulates PTQ as a block-wise optimisation problem, where each block of layers is quantized jointly rather than independently, significantly reducing quantization error propagation. GPTQ [10] applies a one-shot layer-by-layer quantization method using second-order information, and is fast enough to quantize 175-billion-parameter LLMs in a few hours on a single GPU. For sub-4-bit quantization at scale, outlier-aware methods like SmoothQuant [13] and AWQ [14] have emerged, though these target server-class LLMs rather than edge NPUs that support only INT8.

QAT methods like LSQ [11] and PACT [12] learn quantization parameters jointly with model weights during training. LSQ treats the step size as a learnable parameter

with a gradient defined through the Straight-Through Estimator, achieving near-FP32 accuracy at INT4 on ResNets. PACT learns activation clipping thresholds adaptively. Both methods outperform PTQ significantly but require access to the full training pipeline. For edge deployment where training access is available, QAT is generally the better choice—consistent with our recommendation in Section V.

Pruning research has bifurcated between unstructured and structured methods. Unstructured magnitude-based pruning can achieve 90% sparsity with under 1% accuracy loss on large models [8, 9], but relies on sparse-tensor hardware acceleration for practical latency improvement. Network Slimming [15] introduced a structured approach: adding L1 regularisation to batch normalisation scaling factors during training pushes some channels toward zero, then those channels are pruned after training, producing dense models that run fast on any hardware. For Transformers, Michel et al. [6] showed that many attention heads contribute almost nothing to task performance and can be pruned with minimal accuracy loss, while Voita et al. [16] found that surviving heads tend to specialise in linguistically meaningful functions. Both findings support our structured head pruning approach.

Knowledge distillation has grown far beyond Hinton et al.'s original formulation [3]. FitNets [7] introduced intermediate feature map matching as a form of stronger supervision, aligning the student's internal representations to the teacher's at an intermediate layer rather than just at the output—the approach we adopt in our feature alignment loss term. Attention Transfer [17] uses attention maps as the alignment target. Born-Again Networks [18] showed that a student of the same architecture as the teacher can exceed the teacher's performance, an effect attributed to the softer gradient signal from soft labels. In NLP, DistilBERT [19] showed that a 6-layer distilled BERT retains 97% of performance at 40% of the size. TinyBERT [20] extended this with layer-by-layer distillation, achieving even better compression ratios.

The connection between distillation and robustness has been studied in two contexts. Papernot et al. [21] proposed distillation as a defence against adversarial examples, arguing that the softer gradient surface of a distilled model makes it harder to find adversarial perturbations. Hendrycks et al. [22] demonstrated on ImageNet-C that models trained with soft labels from a better model show improved corruption robustness—a finding that motivates our hypothesis that distillation will outperform standalone quantization and pruning on our noise benchmarks. Our work extends this to compressed models in low-data settings, a combination neither paper addresses.

Hybrid and multi-stage compression pipelines have been explored in the neural architecture search literature. Once-for-All [23] trains a single network that can be sliced into sub-networks of different sizes without retraining, covering the joint space of pruning and quantization decisions. ProxylessNAS [24] searches for efficient architectures on the target hardware directly. These methods require substantial compute and expertise. Our hybrid pipeline is deliberately simpler—a fixed three-stage sequence rather than a search—trading potential optimality for reproducibility and accessibility to practitioners with limited compute budgets.

On robustness benchmarking, ImageNet-C [5] and CIFAR-10-C are the standard vision corruption benchmarks. For language, AdvGLUE [25] and ANLI [26] test adversarial robustness but focus on adversarial attacks crafted to fool models, rather than realistic deployment noise like typos and code-switching. To our knowledge, there is no published robustness benchmark specifically targeting code-mixed multilingual text—Hinglish, Marathi-English, or similar. Our noise protocol for these conditions is a small but original contribution to the field.

VIII. CONCLUSION

We started this paper with a simple observation: model compression is evaluated on clean data with abundant labels, but edge AI runs on noisy inputs with tiny per-device datasets. This mismatch means the compression choices that look good on benchmarks may be exactly the wrong choices for deployment. We designed this work specifically to surface that gap and give practitioners the information they need to make better decisions.

Our plan is to compare PTQ, QAT, structured pruning, knowledge distillation, and a three-stage hybrid pipeline across two modalities, four architectures, nine noise types, and three low-data levels, on real embedded edge hardware. Based on our analysis and the patterns in the existing literature, we expect the hybrid pipeline to come out ahead on all the metrics that matter most for deployment: size, latency, power draw, clean-data accuracy, robustness under noise, and performance with limited local training data. Knowledge distillation is the most critical single ingredient—its data-efficiency advantage in low-data regimes and robustness advantage under noise cannot be matched by quantization or pruning alone.

What we think matters more than any specific result is the evaluation framework itself. We are arguing that the field should measure compression quality on noisy inputs and in low-data regimes as a matter of routine, not as a special case. If a compressed model achieves 70% accuracy on clean CIFAR-100 but drops to 38% in a 10-shot noisy setting, that 70% number is misleading for practitioners building real applications. Our MRD and LDP@K metrics are designed to expose these differences, and our noise protocol is designed to be general enough to apply to any model and any compression method. We will release everything—code, checkpoints, noise scripts, and sampling indices—as a public repository.

The federated-style scenario we include is particularly important. When five independent edge nodes, each with only 10 labeled images per class, independently compress the same pretrained model without sharing any data, the 14-point accuracy gap between PTQ and the hybrid pipeline under noise is not a benchmark artefact. It is a realistic

picture of what happens when compression choices are made without considering the conditions of actual deployment. We hope this is the kind of evidence that changes how practitioners think about which compression method to use.

Three directions seem worth pursuing after this work. First, federated compression coordination: can a fleet of edge devices collectively improve their per-device compression decisions by sharing compressed gradients or model structure decisions, while keeping training data entirely local? This would combine the benefits of federated learning with principled compression, a combination that does not yet have a strong literature. Second, robustness-aware per-layer mixed precision: rather than applying QAT uniformly across all layers, can we identify which layers are most responsible for noise fragility—perhaps through gradient sensitivity analysis—and preserve their precision selectively while aggressively compressing the rest? Third, a proper multilingual robustness benchmark covering code-mixed Hinglish, Marathi-English, and similar registers at scale, with standardised noise types and severity levels analogous to what ImageNet-C provides for vision.

Edge AI is not a niche research area anymore. It is where AI needs to actually work for most of the world's population—in CCTV cameras guarding rural clinics without reliable internet, on agricultural drones operating over farms without cloud connectivity, in industrial sensors monitoring equipment on factory floors where a missed alert costs far more than the hardware itself. Getting compression choices right for those conditions—not just for benchmark scores on clean data—is what this paper is working toward, and we hope the evaluation framework is as useful to the community as any specific experimental result we report.

REFERENCES

[1] Y. Li et al., "BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction," in Proc. ICLR, 2021.

[2] T. Hooker et al., "Characterising Bias in Compressed Models," arXiv:2010.03058, 2020.

[3] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015.

[4] S. Cass, "Nvidia's TK1 Mobile Supercomputer," IEEE Spectr., vol. 51, no. 10, pp. 17–18, Oct. 2014.

[5] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions," in Proc. ICLR, 2019.

[6] P. Michel, O. Levy, and G. Neubig, "Are Sixteen Heads Really Better than One?" in Proc. NeurIPS, 2019.

[7] A. Romero et al., "FitNets: Hints for Thin Deep Nets," in Proc. ICLR, 2015.

[8] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal Brain Damage," in Proc. NeurIPS, 1990.

[9] S. Han, H. Mao, and W. J. Dally, "Deep Compression," in Proc. ICLR, 2016.

[10] E. Frantar et al., "GPTQ," in Proc. ICLR, 2023.

[11] S. K. Esser et al., "Learned Step Size Quantization," in Proc. ICLR, 2020.

[12] J. Choi et al., "PACT," arXiv:1805.06085, 2018.

[13] G. Xiao et al., "SmoothQuant," in Proc. ICML, 2023.

[14] J. Lin et al., "AWQ," arXiv:2306.00978, 2023.

[15] Z. Liu et al., "Network Slimming," in Proc. ICCV, 2017.

[16] E. Voita et al., "Analyzing Multi-Head Self-Attention," in Proc. ACL, 2019.

[17] S. Zagoruyko and N. Komodakis, "Attention Transfer," in Proc. ICLR, 2017.

[18] T. Furlanello et al., "Born Again Neural Networks," in Proc. ICML, 2018.

[19] V. Sanh et al., "DistilBERT," arXiv:1910.01108, 2019.

[20] X. Jiao et al., "TinyBERT," in Proc. EMNLP, 2020.

[21] N. Papernot et al., "Distillation as a Defense," in Proc. IEEE S&P, 2016.

[22] D. Hendrycks et al., "The Many Faces of Robustness," in Proc. ICCV, 2021.

[23] H. Cai et al., "Once for All," in Proc. ICLR, 2020.

[24] H. Cai, L. Zhu, and S. Han, "ProxylessNAS," in Proc. ICLR, 2019.

[25] X. Wang et al., "Adversarial GLUE," in Proc. NeurIPS, 2021.

[26] Y. Nie et al., "Adversarial NLI," in Proc. ACL, 2020.